

Interactive Graphics - Project Report

OpenSCUBA: an open-source 3D SCUBA diving experience.

Alessio Maria Falchi

1 Project Overview

The developed project, named *OpenSCUBA*, is an interactive application centered around the exploration of an underwater environment. It draws inspiration from [ARIKA's Endless Ocean](#) videogame series, adopting its characteristic underwater atmosphere and relaxing vibes. The repository is available on [GitHub](#), and the live demo can be accessed directly through a web browser.

The scene features the following main characteristics:

- An **immersive underwater setting** with various scenery elements;
- A **scuba diver character** controllable by the user;
- Several **marine creatures** of various sizes;
- Heads-Up Display (**HUD**);
- **Sound effects** and **music**.

Each one of the aforementioned elements will be discussed more in detail in [Section 3](#).

1.1 How to Run

As already mentioned, the live demo can be run directly from a web browser, without requiring any additional installation or configuration. The latest version is available through the project's [GitHub Pages site](#). Once the demo is opened, a Loading UI will appear, followed by a "Play" button once the assets have loaded. Inside the main scene, the user can interact by using the following controls:

- **Mouse Cursor**: look around the underwater environment;
- **Left Mouse Button**: hold to move the diver;
- **F key**: toggle the flashlight on or off;
- **H key**: show or hide the FPS counter.

These controls allow the user to freely explore the underwater scene and observe the various marine creatures and scenery elements in it.

2 Environment, Libraries and Structure

The project relies on the [Three.js](#) library, along with the following additional libraries:

- [simplex-noise](#) → Used to generate smooth procedural noise for the seabed;
- [three-mesh-bvh](#) → Used to accelerate spatial queries and collision/raycast operations on complex 3D meshes;
- [tween.js](#) → Used to create smooth animations and transitions;

Additionally, the [vite](#) tool has been used to build the environment, and the additional plugin [vite-plugin-static-copy](#) was used to copy static assets to the build output, simplifying the deployment and management of the required resources. Ultimately, a GitHub deployment action has been configured, in order to ensure that the application is automatically built and deployed whenever changes are pushed to the repository.

Assets Used All the assets (3D models, textures, sounds) used for this project have been either downloaded from third-parties or AI-generated. Further information about each asset can be found in the repository's README file.

2.1 Folders Structure

The project folders have been organized as follows, prioritizing clarity and modularity:

- **assets** → Contains 3D models, organized into subfolders for easier file management;
- **components** → Contains JavaScript classes that define the elements of the scene and their behaviour (e.g. water, sun, creatures, etc.);
- **core** → Contains the main JavaScript scene files and the controls configuration;
- **docs** → Contains the project's documentation files, such as this technical report;
- **shaders** → Contains classes implementing the GLSL shaders;
- **sounds** → Contains sound effects and music files;
- **textures** → Contains texture files;
- **ui** → Contains JavaScript classes responsible for handling UI behaviour;
- **utils** → Contains reusable JavaScript utility classes.

2.2 Resources Management

To limit memory leaks and consequent RAM fillup, especially during the debugging phase, a `GarbageCollector` class was declared inside the `/utils/garbage_collector.js` module. The latter includes two static methods to correctly dispose meshes, materials and textures previously loaded inside the scene. Moreover, since laptop devices often rely on hybrid GPU configurations and tend to use the integrated GPU for browser applications in order to limit power consumption, the `'high-performance'` value has been passed to the `powerPreference` parameter of the WebGL renderer when the latter is instantiated, so that the browser is encouraged to use the dedicated GPU instead, if available.

3 Scene Design and Implementation

3.1 Core Modules and Setup

All the parts that will be described in the following (sub)sections are brought together by the module `/core/world.js`: the latter declares the `World` class, whose constructor instantiates the scene, the renderer, the camera, the (ambient) lighting, and all the scenery elements. Additionally, shadow mapping is globally enabled on the WebGL renderer using Percentage-Closer Filtering (`PCFShadowMap` class). This ensures soft, realistic shadow projections across the scene models when illuminated by primary light sources. The class also contains a `render` function, which updates all the "dynamic" components by getting called at every frame (e.g. world time, animations, etc.), and some utility functions to help with application start, class deinstantiation, and collision detection (more about this in Section 3.4.3). The `World` class is instantiated by the `/core/main.js` script, which is executed by `index.html`. The `index.html` file defines the webpage structure and includes the Loading UI configuration, styled via an embedded CSS `style` tag. The Loading UI's behavior is handled by the `LoadingUI` class, defined in the `/ui/loading-ui.js` module and instantiated by `main.js`. Once the loading is complete and the user has clicked on the "Play" button, the `/core/controls.js` module is enabled. The latter declares a `Controls` class, which changes the camera position and orientation according to some properly configured event listeners (mouse cursor movement and left mouse button click). Moreover, some additional event listeners have been added in order to pause the camera movement in case the browser window loses focus or the mouse cursor goes out of it.

3.2 Models and Textures loading

To simplify the loading of models and textures within the scene, two utility classes, `ModelLoader` and `TextureLoader`, are declared in the `/utils/model_loader.js` and the `/utils/texture_loader.js` modules, respectively.

Model Loader The `ModelLoader` class instantiates a `GLTFLoader` in its constructor and configures it to use the `MeshoptDecoder` (useful for compressed models). Its `loadModel` method takes the model `path` as an argument and returns a `Promise` containing both the loaded model and its animation clips¹. The class also provides a `setupAnimation` helper method for creating an `AnimationMixer` and playing a selected animation.

Texture Loader The `TextureLoader` class is implemented as a static utility class and therefore cannot be instantiated. Its `load_texture` method loads a texture from the specified path and allows the caller to configure its repeat value, color space according to the texture type, and wrapping mode independently for the horizontal and vertical axes. If no wrapping mode is specified, `MirroredRepeatWrapping` is used for both axes.

If not differently or explicitly specified, any model or texture import mentioned in this report has been performed by taking advantage of these utility classes.

¹Although the utility class allows it, third-party animations have **not** been used for this project.

3.3 Underwater Environment

The scene setting is made of 4 main components: the seabed, the water, the sun and the scenery elements.

3.3.1 Seabed

The **Seabed** class is defined in the `/components/seabed.js` module and consists of an 800x800 **PlaneGeometry**. To enhance its realism, a subtle noisy effect was applied to the plane using the `createNoise2D` function, imported from the `simplex-noise` library. The vertex normals are then recomputed to ensure that light is reflected correctly across the deformed surface. A custom sand-colored **MeshStandardMaterial** was applied to the plane, using a soil texture along with a wavy sand texture for the normal, roughness, and ambient occlusion maps. These sand textures are configured to use **RepeatWrapping**, allowing them to cover the entire surface seamlessly. Finally, the seabed is configured to receive shadows from other objects in the scene.

3.3.2 Water

The main underwater effect is achieved through cyan ambient lighting and renderer color, combined with dense fog (an instance of the **Fog2Exp** class), all configured in the `world.js` file. Similarly to the seabed, the water surface is defined in the **Water** class of the `/components/water.js` module as an 800x800 **PlaneGeometry**. However, this time a **ShaderMaterial** was used, along with custom GLSL for Vertex and Fragment Shaders, defined in the **WaterShader** class, inside the `/shaders/water_shader.js` module.

Vertex Shader The Vertex Shader relies on the Gerstner wave function, which moves each point along a small circular/elliptical path, providing a more realistic ocean-like movement than a simple sine wave. More in particular, four Gerstner waves (whose configuration details are reported in Table 1) were summed together in order to further increase realism.

Wave	Direction	Steepness	Wavelength	Role
1	(1.0, 0.3)	0.05	20.0	Broad primary swell
2	(0.3, 1.0)	0.035	14.0	Secondary swell
3	(-0.7, 0.5)	0.02	9.0	Fine surface detail
4	(0.2, -0.8)	0.01	6.0	Fine surface ripple

Table 1: Characteristics of the wave components.

The amplitude of each wave is not set directly, but derived from its steepness and wavelength according to the formula

$$A = \frac{SL}{2\pi} \quad (1)$$

where $S \in [0, 1]$ is a dimensionless steepness factor and L is the wavelength.

In addition to displacing each vertex, the Vertex Shader also recomputes the surface normal at every point, since the wave displacement continuously changes the surface’s local orientation. Rather than differentiating the wave function analytically, the normal is approximated numerically: the displaced position is sampled at two infinitesimally offset

points along the x and z axes, and the resulting tangent vectors are combined via cross product to obtain the local normal. This normal, together with the vertex's elevation and world position, is then passed to the Fragment Shader.

Fragment Shader The Fragment Shader computes each pixel's color using the interpolated normal, elevation, and world position received from the Vertex Shader, together with the current position, color and intensity of the sun light source, which vary throughout the day-night cycle. In particular, it features:

- An approximated Fresnel effect, which describes how the reflectivity of the water surface changes based on the viewing angle;
- Pixel coloring based on its specific elevation;
- A Blinn-Phong lighting model, combining ambient, diffuse and specular components to shade the surface according to the sun's direction.

All the aforementioned lighting effects are then blended with the sky color via the Fresnel term to simulate reflection, where the sky color itself is the renderer's clear color (passed to the shader as the `uSkyColor` uniform) blended at 40% with a fixed reference tone so that the reflected surface stays visually distinct from the actual sky. The Fresnel effect has been capped to 70% in order to avoid the possibility of full mirror-like reflections occurring.

To simulate the water depth effect, both the fog density and color are dynamically adjusted in the world `render` function according to the current frame's camera position. More in particular, if the camera's Y position² is higher than a certain threshold the fog color gets progressively lighter and less dense through the use of `lerp` functions (one for each parameter to linearly interpolate).

3.3.3 Sun

The `Sun` class in the `/components/sun.js` module defines the sun as a `SphereGeometry` instance. A `DirectionalLight` with shadow casting enabled is attached to the sun and moves with it. Additionally, a slight glowing effect, created with a `CanvasTexture`, is applied to the sphere to improve realism. The `update` method inside the class is called by the world `render` function and ensures that both the directional light and the glowing effect follow the sun. Moreover, a simple daylight cycle dynamically updates the sun's position and color according to the time of day, along with the renderer's clear color. The day duration (in seconds) is defined in the main world module and is set to 1200 (= 20 minutes).

3.3.4 Scenery

Any 3D model used solely for decorative purposes in the underwater environment is classified as a scenery object. To represent these objects, a `Scenery` class was defined in the `/components/scenery.js` module. In addition to the model path, the class constructor accepts `scale`, `position`, and `orientation` parameters. It also allows the model to be configured to either collide with the scuba diver or remain non-collidable. Finally,

²It is relevant to remind that the Y axis in `Three.js` corresponds to the height/vertical one (normal to the horizontal XZ plane).

a custom texture can be assigned to the model; if no custom texture is provided, the texture embedded in the model is loaded automatically, if available. For some models, lower-resolution textures were used for a faster loading time. Some auxiliary functions (`ensureLitMaterial`, `normalizeScanMaterial`, and `generateTriplanarUVs`) have been created to adapt models whose materials do not properly respond to scene lighting or whose geometry does not already provide UV mapping for texture application.

In order to place multiple objects with ease, a `SceneryWrapper` class has been created in `/utils/scenery_wrapper.js`. Instancing groups of scenery objects with this class allows to easily move and scale them all with respect to a common reference frame, while still maintaining the possibility of setting up specific properties for each of them.

The inserted scenery elements are the following:

- **Rock walls:** define the boundaries of the world while preserving a realistic visual appearance;
- **Rock mountain:** features a custom color texture;
- **Shipwreck:** serves as a major underwater scene element;
- **Artificial reef:** provides an additional man-made structure within the environment;
- **Coral reef:** consists of a base rock with a custom color texture and several coral models placed on top using `SceneryWrapper`;
- **Kelp plants:** use a custom solid-green color texture, are placed in batches using `SceneryWrapper`, and have collisions disabled (realism compromise choice).
- **Machinery:** a futuristic device that emits a bright light beam during night time.

About the Machinery's Light Beam The machinery light beam, instance of the `Beam` class defined in `/components/beam.js`, consists of a very tall `ConeGeometry` and a bright `SpotLight`. To illuminate the machinery base as well, an additional light-blue coloured `PointLight` (without shadow casting) has been added. The light's on/off transition is animated using the `Tweener` utility abstract class, defined in `/utils/tweener.js`, which relies on the `tween.js` library.

3.4 Scuba Diver

The Scuba Diver (`ScubaDiver` class in the `/components/scubadiver.js` module) is the main character of the scene and is controlled by the user.

3.4.1 Model and Animation

A third-party scuba diver 3D (hierarchical) model has been imported and its animations have been rewritten from scratch. The model uses a Mixamo-style³ skeleton, from which a fixed set of bones (spine, head, upper and lower arms, upper and lower legs, and feet) is extracted and stored by name for direct manipulation each frame. Rather than relying on the skeleton's default pose, the bind (rest) rotation of every bone is reconstructed

³Mixamo is a free, web-based platform that allows users to automatically rig 3D character models.

from the skeleton’s inverse bind matrices; this rest pose serves as the neutral baseline around which all procedural animation is applied. Two states are blended together at every frame: an *idle* state, in which the diver drifts gently in place, and a *swimming* state, triggered when the player moves forward. A single blend factor is smoothed over time using exponential interpolation, so that transitions between idle and swimming feel gradual. Within each state, limb rotations are driven by sine and cosine functions offset by a quarter period, producing a natural alternating swing-and-bend cycle. Paired bones (left/right) are mirrored, and the amplitude of each motion is larger in the swimming state than in the idle state, with the overall animation speed tied to the player’s movement speed.

3.4.2 Camera Attachment and Controls

As described in Section 3.1, the `controls.js` module updates the camera orientation according to the mouse cursor position, while the left mouse button controls forward movement. The camera pitch is bounded to $\pm(\pi/2 - 0.05)$ to prevent it from flipping upside down. The scuba diver model subsequently follows the camera, with a `localOffset` transformed according to the camera orientation in order to maintain a third-person view. The model orientation is derived from the camera orientation and the mouse cursor position, with the body pitch and cursor-dependent rotations bounded to prevent unnatural movements. The diver’s head is also independently rotated in pitch and yaw according to the cursor position, allowing it to partially follow the cursor.

Flashlight The diver features a flashlight that the user can switch on and off by pressing the F key (default state is off). The flashlight is implemented as a `SpotLight` attached to the front of the diver model and is configured with a high intensity (100), a relatively short maximum range (30), and a relatively wide cone angle ($\frac{\pi}{3} = 60^\circ$). Moreover, shadow casting is enabled for the flashlight, so its illumination can produce dynamic shadows on objects within its range that are configured to receive them.

3.4.3 Collision Detection

The diver’s movement through the environment is constrained by two independent collision systems:

- **resolveCreaturePushback** → Handles dynamic objects marked as collidable creatures. A world-space axis-aligned bounding box is computed for each object mesh and padded by a fixed collision offset. If the diver’s collision position falls inside a box, signed overlap distances are calculated along each principal axis, and the diver is pushed out along the axis requiring the minimal displacement to prevent the player from penetrating or getting stuck inside bounding volumes.
- **resolveSceneryPushback** → Handles static scenery: for objects marked as collidable and equipped with a bounds tree⁴, the closest point on the object’s surface to the diver is computed in the object’s local space. If this point lies closer than the diver’s collision radius, the diver is pushed outward along the surface normal by the amount of overlap.

⁴The bounds tree is computed by taking advantage of the `three-mesh-bvh` library.

Moreover, hard bounds have been set to prevent the diver from getting either below the seabed or above the water surface. This is handled by the `resolveEnvironmentCollisions` auxiliary function in `world.js`, which computes the diver’s world-space position from the camera position and a provided offset and then applies the two constraints independently. The seabed constraint relies on a `Raycaster` instance cast downward from above the diver toward the seabed mesh: the resulting hit point gives an accurate local distance from the seabed, adapting to its noise-generated irregularities discussed in Section 3.3.1, and the camera is pushed upward if the diver’s Y position falls below this height plus a clearance margin. The water-surface constraint is simpler: it clamps the diver’s Y position so that it never exceeds the water level minus a specified clearance, pushing the camera downward if this bound is violated.

3.5 Marine Creatures

The marine creatures system is organized across two main module structures:

- `/components/marinecreatures.js` → Implements shared properties and methods via the abstract `MarineCreature` class and its subclasses (`SmallMarineCreature`, `LargeMarineCreature`).
- `/components/creatures/<creaturename>.js` → Defines individual `<CreatureName>` classes inheriting from their respective superclass.

The `MarineCreature` class sets up the utilities to efficiently load a creature model and the basic variables to keep track of the state for its animation (e.g. elapsed time, idle/swim blend, etc.). Like most scenery objects, creature models use textures imported directly from their `.glb` files. In some cases, lower-resolution textures were selected to improve load times.

3.5.1 Small Marine Creatures

Subclasses of the `SmallMarineCreature` abstract class include all the creatures smaller than the scuba diver model. This type of creatures features a *wandering* algorithm which works as follows:

1. **Initialization:** Upon creation, the creature is configured with a wandering volume, a base speed, and an idle duration range, and it immediately enters a randomized idle state while selecting its first destination.
2. **Target selection:** Destinations are chosen randomly within the wandering bounds using a compressed vertical axis, which encourages predominantly horizontal swimming.
3. **Behavior update:** During each simulation step, the creature either gradually coasts to a complete stop for a randomized idle period, or travels toward its current destination while smoothly decelerating on approach.
4. **Orientation:** Rather than snapping abruptly, the creature utilizes spherical linear interpolation to continuously and smoothly rotate toward its direction of travel.

5. **Animation feedback:** The logic calculates and returns a normalized ratio of the creature's current speed against its base speed, yielding a numeric value suitable for driving animation blends.

The small creatures that have been imported inside the scene are the following:

- **Coral Fish:** simple model, with a red colored texture;
- **Emperor Angelfish:** hierarchical model, re-animated from scratch to take full advantage of its rigged structure, with fin movements varying according to its wandering speed.

Since small marine creatures realistically tend to occur in high numbers and in close proximity to one another, a `CreatureWrapper` class has been declared in the `/utils/creature_wrapper.js` module, in order to make the spawning of large numbers of creatures easier and less verbose. The wrapper also implements a maximum distance threshold for each creature's shadow casting, resulting in a significant overall performance increase. Moreover, in order to make the diving experience more fluid and enjoyable, collisions for small creatures have been kept disabled.

3.5.2 Large Marine Creatures

Subclasses of the `LargeMarineCreature` abstract class include all the creatures larger than the scuba diver model. Differently from the small creatures discussed in the previous section, large creatures don't feature a random wandering algorithm, but their trajectories are instead hard-coded and vary according to the specific creature. More in particular, the large creatures imported inside the scene are the following:

- **Whale Shark:** hierarchical model following a circular path at constant speed with a horizontally swinging caudal fin;
- **Manta Ray:** hierarchical model moving along an ellipsoidal trajectory at constant speed with symmetrically flapping pectoral fins;
- **Killer Whale:** hierarchical model following a linear trajectory⁵ with a vertically oscillating caudal fin.

3.6 Heads-Up Display (HUD)

Some dynamic HUD elements have been added to the scene to make the experience more immersive. Each element (except for the FPS counter, which was instantiated directly in the `World` class constructor) was implemented as a separate `<ElementName>` class inside its respective `/ui/<element_name>.js` model.

The added elements are the following:

- **Compass:** Located in the top-left corner of the screen, it displays the camera's yaw both graphically and as a numerical value in degrees.
- **Depth Gauge:** Located in the bottom-left corner of the screen, it displays the scuba diver's current depth relative to the water surface, both graphically and as a numerical value in meters.

⁵Implemented by collapsing the vertical component of the ellipsoidal trajectory function.

- **Time Display:** Located in the bottom-right corner of the screen, it displays the current world time in `<hh:mm>` format.
- **FPS Counter:** An instance of `Three.js`'s `Stats` class located at the top-left of the screen, which displays the real-time FPS and can be toggled by pressing the `H` key (hidden by default).

3.7 Sound Effects and Music

In order to make the interactive experience more engaging, audio has been inserted inside the scene. An `AudioManager` class, declared in the `/utils/audio_manager.js` module, takes care of loading, playing, and stopping the specified audio files (`load`, `play` and `stop` methods). It is possible to specify the desired volume for each sound, whether it should loop, and whether it should play globally or at a specific 3D position, defined by a mesh target. The following sounds have been inserted in the scene:

- **Play button** click sound effect, followed by a **splash** sound while transitioning from the loading UI to the underwater scene;
- **Underwater ambience** loop;
- **Scuba diver breathing** sound, played from the diver model at low volume;
- **Flashlight click** sound, played from the diver model whenever the flashlight is turned on or off;
- **World music**, played only once after the user has entered the underwater scene.

4 Conclusion

OpenSCUBA brings together a fully explorable underwater environment, a procedurally animated scuba diver, a small ecosystem of wandering and path-following marine creatures, and supporting HUD and audio systems into a single cohesive browser-based experience. The most technically demanding aspects were the water shader, which combines multiple Gerstner waves with custom Fresnel-based shading, and the animation of the scuba diver, which had to blend idle and swimming states smoothly.

The project has been developed with modularity in mind, potentially allowing an easier integration of future extensions, such as a wider variety of marine creatures and scenery, multiple environments (e.g. abyss, river, etc.) and richer diver-environment interactions. Overall, the project met its goal of delivering a relaxing, immersive underwater exploration experience directly playable from a web browser.

About AI Usage Generative AI has been used for the following purposes:

- Code writing support (GitHub Copilot, ChatGPT, Claude, Gemini);
- Sentences polishing and rephrasing to improve clarity and readability (ChatGPT, Claude, Gemini);
- Generation of specific assets (see the repository `README` file).